

CASE STUDY 2 — Global bank

Lead Data Engineer · Sep 2024 – Oct 2025 · AZURE Cloud · Banking Domain

STAR — PROJECT OVERVIEW

Situation

Global bank's global trade finance and risk analytics function relied on a legacy Oracle-based estate with nightly Informatica ETL to join trade-finance transactions, counterparty hierarchies, credit limits, collateral positions, and exposure data from core banking systems. Regulatory reporting for FCA/PRA reconciliation was largely spreadsheet-driven, with manual Excel macro-based validation performed each reporting cycle — a process prone to human error and with no reliable audit trail.

Risk exposure figures were only ever T+1 accurate: because the batch ran overnight, intraday counterparty and market-risk exposure had no automated calculation path, forcing the Risk team to work from stale figures for anything requiring a same-day view. Regulatory transformation logic for Basel III and FRTB calculations lived scattered across Informatica mappings and manual spreadsheet formulas with no versioning or lineage — an FCA audit could take weeks to trace a submitted figure back to its source.

The technical decision-making function I supported needed a platform that could deliver reconciled, auditable trade-finance datasets on a reliable schedule, close the intraday exposure gap for Risk Operations, and replace the spreadsheet-driven reconciliation with a governed, lineage-traceable process suitable for FCA/PRA scrutiny.

Task

As Lead Data Engineer, with two data engineers reporting into me, I was responsible for designing and building the Azure Databricks PySpark pipelines joining trade-finance and risk datasets from core banking systems; for the near-real-time trade-risk processing flow used by Risk Operations; for the Delta Lake regulatory transformation layers underpinning Basel III/FRTB calculations; and for re-engineering the FCA/PRA reconciliation workflow using Microsoft Fabric. I also supported technical decision-making and design reviews across the wider platform, evaluating approaches for performance, scalability, reliability, security and cloud cost.

Action

I developed Azure Databricks PySpark pipelines that join trade-finance transactions, counterparty hierarchies, credit limits, collateral positions, and exposure data from core banking systems, publishing reconciled datasets to ADLS Gen2 for the Risk and Regulatory Reporting teams to consume.

I built a near-real-time trade-risk processing flow using Kafka (via Azure Event Hubs) and PySpark Structured Streaming — applying event validation, deduplication, and watermarking before calculating intraday counterparty and market-risk exposure, closing the T+1 gap for Risk Operations.

I designed the regulatory transformation layers in Delta Lake supporting Basel III and FRTB calculations, implementing the business-rule transformations in PySpark and SQL together with Delta MERGE and historical versioning to give the regulatory reporting datasets a fully auditable change history.

I re-engineered the FCA/PRA reconciliation workflow using Microsoft Fabric — building source freshness checks, relationship tests, reconciliation models, and exception datasets that replaced the previous spreadsheet-driven validation entirely.

I implemented orchestration using Airflow dependency graphs together with Azure Data Factory, deliberately separating ingestion, regulatory transformation, reconciliation, and publication into distinct workflows rather than one monolithic pipeline, so a failure in one stage never blocks or silently corrupts another.

I configured Unity Catalog and Microsoft Fabric workspace integration — catalog/schema segregation, column-level classifications, lineage metadata, and access controls for sensitive counterparty and financial data — and built reusable Terraform modules for Databricks clusters, ADF linked services, Synapse serverless resources, and Event Hub namespaces to support consistent CI/CD deployment.

Result

- Closed the risk exposure gap from T+1 to intraday (sub-5-minute) for counterparty and market-risk figures via the real-time trade-risk flow
- Eliminated the spreadsheet-driven FCA/PRA reconciliation process, cutting reconciliation cycle time from roughly 2 weeks to same-day with a full audit trail
- Reduced regulatory-figure audit turnaround from multi-week manual tracing to under 3 days using Unity Catalog and dbt-documented lineage

- Achieved zero FCA restatements attributable to the new platform since go-live, down from a previously error-prone manual process
- Cut new-environment provisioning time from roughly 2 weeks to under a day via reusable Terraform modules
- Supported design reviews that materially reduced projected cloud cost on two major workstreams by right-sizing Databricks cluster and Synapse serverless configuration

ARCHITECTURE OVERVIEW

The platform runs two clearly separated tracks that share a common storage layer: a batch track carrying the full analytical and regulatory workload, and a supplementary real-time track for the specific low-latency use case below. They never share pipeline code — only the storage layer.

Batch Track

```

SOURCE SYSTEMS (core banking systems, counterparty master, collateral systems, credit
limit engine)
    ↓ [Azure Data Factory — Batch Orchestration & Data Movement]
BRONZE LAYER — ADLS Gen2 (raw zone) — Delta Lake format
    ↓ [Azure Function — Schema Evolution Check]
    ↓ [Databricks PySpark — Silver cleansing, joins, reconciliation]
SILVER LAYER — ADLS Gen2 (curated zone) — regulatory transformation (Basel III /
FRTB), Delta MERGE + versioning
    ↓ [dbt — Gold business models]
GOLD LAYER — Azure Synapse Analytics — Star schema (fact + dim), Microsoft Fabric
reconciliation models
    ↓
CONSUMERS: Risk & Regulatory Reporting teams · Power BI · FCA/PRA submissions
           Data Science (reads Gold Delta directly via Databricks notebooks)

ORCHESTRATION: Apache Airflow (dependency graphs) + Azure Data Factory (data movement) —
separate roles
  
```

Real-Time Track (separate track)

```

TRADE EVENT SOURCE (core banking trade capture system)
    ↓
Apache Kafka (Azure Event Hubs, Kafka-compatible protocol; topics: raw → enriched → dlq)
    ↓ [PySpark Structured Streaming on Databricks — validation, dedup, watermarking]
    ↓
├── ADLS Gen2 (real-time zone — permanent record)
│   (merged by next batch run)
└── Azure CosmosDB (NoSQL hot path)
    Sub-5ms reads
    → Intraday exposure dashboard
  
```

SECTION A — BATCH PIPELINE (PRIMARY)

Start here in the interview. This is the core of the platform and should be walked through in full before any mention of real-time.

Step 1: Data Sources & Ingestion

Trade-finance transactions, counterparty hierarchies, credit limits and collateral positions arrive from core banking systems and reference data feeds via Azure Data Factory. Reference data (counterparty master, product hierarchy) is loaded full-refresh; transactional data (trades, exposure movements) is loaded incrementally against a watermark held in an Azure SQL Database control table.

All connection credentials are stored in Azure Key Vault — never hardcoded — and referenced by ADF linked services and Databricks secret scopes.

Ingestion tool: Azure Data Factory (Copy Activity) + Databricks PySpark

Pattern: Full load (reference) + watermark incremental (transactional)

Credential store: Azure Key Vault

Step 2: Bronze/Raw Layer

Data lands in ADLS Gen2 raw zone in Delta Lake format exactly as received, with `ingested_at`, `source_system`, and `pipeline_run_id` audit columns, partitioned by `load_date` and `source_system`.

Delta Lake was the deliberate choice here — not plain Parquet — because it gives ACID transactions for safe concurrent writes from both ADF batch loads and the streaming sink, plus time-travel, which was directly useful when tracing a Silver defect back to the exact Bronze state that produced it.

Format: Delta Lake on ADLS Gen2

Retention: Raw zone kept 90 days before archive tier

Step 3: Schema Evolution (post-Bronze)

An Azure Function, triggered by an ADF pipeline completion event via Azure Event Grid, compares the newly landed schema against the schema recorded in Azure Purview.

- Additive change (e.g. core banking adds a new collateral type field): auto-approved, Purview schema entry updated, Delta mergeSchema enabled for the next Silver write, pipeline continues.
- Breaking change (e.g. a counterparty credit rating field changes from a string code to a numeric scale): the batch is moved to an ADLS Gen2 /quarantine/ prefix, a flag is written to the Azure SQL control table, and an alert is published via Azure Service Bus to the on-call Slack channel — Silver skips that source until reviewed.



INTERVIEW TIP

How to answer "How did you handle unexpected schema changes?" — The Azure Function runs in under 10 seconds and adds no meaningful pipeline latency, but it caught four production-impacting schema changes from upstream core banking releases in six months — all four would previously have silently corrupted regulatory transformation logic without it.

Step 4: Silver/Curated Transformation

Databricks PySpark jobs join trade-finance transactions against counterparty hierarchies, credit limits, and collateral positions, applying type standardisation, null-to-DLQ handling, business-rule mapping (product/status codes), deduplication on `trade_id` + `version`, and derived risk metrics.

The regulatory transformation layer for Basel III and FRTB calculations sits here too, implemented in PySpark and SQL with Delta MERGE for upserts and Delta's built-in historical versioning giving every regulatory-relevant change a fully auditable trail — a Scala Spark component with a custom Partitioner was used for the highest-volume counterparty reconciliation job, eliminating an expensive shuffle across 3M+ records.

Step 5: Gold/Serving Layer (dbt)

dbt models transform Silver Delta tables into fact and dimension tables in Azure Synapse Analytics, structured staging → intermediate → marts, with 150+ dbt tests on every build and dbt docs published as the lineage reference for FCA/PRA audits.

Macros used: `generate_surrogate_key`, `safe_divide` (loss/utilisation ratios), `date_spine` (regulatory calendar)

Step 6: Orchestration

Apache Airflow dependency graphs sit above Azure Data Factory: ADF is the execution engine that actually moves data between systems and connects to core banking and collateral APIs; Airflow is the scheduler and cross-system dependency manager that ensures ingestion, regulatory transformation, reconciliation and publication run as four distinct, ordered workflows rather than one monolithic pipeline.

This separation was a deliberate design decision: previously a single failure anywhere in the monolithic Informatica flow blocked everything downstream. With four independently orchestrated stages, a reconciliation-stage issue no longer blocks ingestion of the next day's trades.

Orchestrator: Apache Airflow (dependency graphs)

Data movement: Azure Data Factory (execution engine, separate role from Airflow)

Retry policy: 3 retries, exponential back-off, SLA alert to Slack then PagerDuty

INTERVIEW TIP

How to answer "What's the difference between ADF and Airflow?" — ADF is the workhorse with 100+ native connectors that physically moves and transforms the bytes; Airflow is the conductor that decides sequencing and dependency across entirely separate systems (ADF, Databricks, Fabric, dbt) that ADF alone has no visibility into. Airflow DAGs are Python in Git — testable and peer-reviewed — where ADF pipelines on their own are GUI-authored and much harder to unit test.

Step 7: Data Warehouse & Modelling

Gold is a star schema in Synapse Analytics: FACT_TRADES (transactional grain, one row per trade) and FACT_CREDIT_EXPOSURE (periodic snapshot, one row per counterparty per product per day) joined to DIM_COUNTERPARTY, DIM_INSTRUMENT, DIM_TRADER and DIM_DATE.

DIM_COUNTERPARTY is modelled as SCD Type 2, because the credit rating and country-of-risk attached to a counterparty at the time of a trade is exactly what FCA requires for capital calculation traceability — overwriting the rating in place would make it impossible to reconstruct historical risk-weighted asset figures.

- Surrogate keys are generated with `dbt_utils.generate_surrogate_key` for every dimension — LEI codes and counterparty IDs from source systems are strings and occasionally reformatted, which breaks natural-key joins.
- Fact rows carry the surrogate key of the counterparty version active when the trade was processed, so downstream queries never need a runtime BETWEEN date filter — a plain equi-join always returns the historically correct credit rating.

INTERVIEW TIP

How to answer "How do you handle slowly changing dimensions?" — Walk through DIM_COUNTERPARTY SCD2: a credit downgrade inserts a new row with a new surrogate key rather than overwriting the old one, the prior row gets `eff_end_date` set and `is_current=FALSE`, and the fact table always joins on the surrogate key assigned at trade-processing time — giving FCA a point-in-time-correct view without any runtime date logic.

Step 8: Data Quality Framework

Great Expectations suites validate Bronze data before any transformation runs — volume thresholds, mandatory-field null checks on `trade_id/counterparty_id`, and referential integrity between trades and counterparty reference data.

dbt tests validate Gold — uniqueness on surrogate keys, `not_null` on mandatory fields, relationships between fact and dimension tables, and custom range checks (e.g. `notional_usd` within plausible bounds). Every Gold publish reconciles total notional back to the source system control total; a variance above 0.01% blocks publication and pages the regulatory reporting lead.

Step 9: Security & Data Governance

Unity Catalog and Microsoft Fabric workspace integration enforce catalog/schema segregation, column-level classification, lineage metadata, and RBAC for sensitive counterparty and financial data. Three access tiers apply: platform admin (Terraform-provisioned, full access), data engineers (read/write Silver and Gold, no Bronze modification), and data consumers (read-only Gold, specific columns only).

PII and commercially sensitive counterparty fields are column-masked in Unity Catalog for all roles except Compliance; encryption at rest uses Azure Key Vault-managed keys, encryption in transit is TLS 1.2+, and all data access is audit-logged with 7-year retention for FCA requirements.

INTERVIEW TIP

How to answer "How did you handle GDPR/FCA compliance?" — Cover column-level masking in Unity Catalog, KMS-managed encryption at rest, 7-year audit log retention, and the dbt docs + Purview lineage that lets an FCA auditor trace any figure in a submission back through every transformation to its Oracle source — the audit process went from roughly 3 weeks to 3 days.

Step 10: How Data Reaches Consumers

Power BI connects to Synapse Analytics Gold tables via JDBC for the Risk and Regulatory Reporting teams. Data scientists read Gold Delta tables directly from ADLS Gen2 via Databricks notebooks, keeping ML workloads off the shared warehouse. FCA/PRA regulatory submissions run through a dedicated Airflow task that reads Gold, applies the Microsoft Fabric reconciliation model, and only submits once the reconciliation and exception checks pass clean.

SECTION B — REAL-TIME PIPELINE (SUPPLEMENTARY)

Only raise this if asked. It is a parallel, smaller-scope track solving one specific latency problem — it does not replace the batch pipeline.

Why Real-Time is Needed

The overnight batch meant intraday counterparty and market-risk exposure had no automated figure — Risk Operations needed the ability to see exposure move as trades were booked, specifically to catch a limit breach before end of day rather than discovering it the next morning when the position could no longer be unwound cheaply.

Event Hub / Kafka Configuration

Azure Event Hubs with the Kafka-compatible protocol carries trade events — the existing Kafka consumer code needed zero changes to point at Event Hubs. Topics: raw → enriched → dlq, replication factor 3, Avro via the Event Hubs Schema Registry.

Broker service: Azure Event Hubs (Kafka protocol)

Topic sizing: ~300K trade/risk events/hour ≈ 83 msgs/sec → 16 partitions

Consumer groups: One per downstream consumer (dashboard, CosmosDB writer, batch merge)

Stream Processing

PySpark Structured Streaming on Databricks applies event validation, deduplication on trade_id + event_version, and a 10-minute watermark for late events, before calculating intraday counterparty and market-risk exposure via a windowed aggregation against a broadcast-joined credit limit reference table. Checkpointing to ADLS Gen2 gives exactly-once recovery after any restart.

NoSQL Hot Path — Azure CosmosDB

The intraday exposure dashboard needs sub-5ms point reads of "what is counterparty X's current exposure" — a query pattern Synapse Analytics is not built for at that latency and concurrency. CosmosDB (Core/SQL API) is written to alongside the ADLS Gen2 real-time zone: partition key /counterparty_id, Session consistency (so a write is always visible to the writer's own subsequent read), RU throughput provisioned for peak trade booking volume.

INTERVIEW TIP

How to answer "Why NoSQL instead of your data warehouse?" — Synapse is optimised for complex analytical SQL over large historical datasets; CosmosDB gives sub-5ms single-key reads at high concurrency, which is exactly what a live exposure dashboard refreshing every few seconds for many risk desks simultaneously needs. Using Synapse for that workload would mean 100ms+ latency per lookup and connection-limit contention under concurrent dashboard load.

Connection Between Batch and Real-Time

Both tracks write to the same ADLS Gen2 account but different prefixes. The daily batch pipeline reads the standard raw prefix plus the real-time zone for the same day, deduplicates on trade_id, and processes both through the same Silver/Gold regulatory transformation path — so Synapse Gold always has the complete, reconciled figure for FCA reporting, while CosmosDB only ever serves the "right now" dashboard read.

SAMPLE DATA

FACT TRADES

trade_sk	trade_id	trade_date	counterparty_fk	notional_usd	rwa_amount	trade_status
441021	TRD-001142	2026-09-18	10089	5,250,000	3,937,500	CONFIRMED
441022	TRD-001143	2026-09-18	10042	12,750,000	9,562,500	PENDING
441023	TRD-001144	2026-09-18	10043	3,100,000	620,000	SETTLED
441024	TRD-001145	2026-09-17	10044	8,900,000	890,000	AMENDED
441025	TRD-001146	2026-09-17	10089	6,400,000	5,120,000	CONFIRMED

DIM_COUNTERPARTY (SCD Type 2)

counterparty_sk	counterparty_id	name	credit_rating	eff_start_date	eff_end_date	is_current
10041	CPT-1042	Goldman Sachs Intl	AA	2024-01-01	2026-06-14	FALSE (credit downgrade)
10089	CPT-1042	Goldman Sachs Intl	AA-	2026-06-15	9999-12-31	TRUE (current version)
10042	CPT-0817	Morgan Stanley & Co	A+	2023-03-01	9999-12-31	TRUE
10043	CPT-2031	Deutsche Bank AG	BBB+	2022-07-01	9999-12-31	TRUE

Real-Time Event — Field Table

Kafka/Event Hub (Kafka protocol) topic: Global bank.traderisk.events.raw

Field	Type	Example
trade_id	string	TRD-001142
event_version	integer	2
counterparty_id	string	CPT-1042
notional_usd	decimal	5250000.00
exposure_delta_usd	decimal	+120000.00
risk_indicator	string	NORMAL (NORMAL/ELEVATED/BREACH)
submitted_ts	timestamp	2026-09-22T08:42:17.000Z
channel	string	CORE_BANKING_API